Onyx Engine

PRODUCT SHEET

A single-model, OpenAI-compatible server built on llama.cpp.

Chat, completions and embeddings from one GGUF or safetensors model, with streaming, continuous batching, tool calls and structured output.

What ships

Continuous batching

--parallel N slots share one batch per tick, so requests generate together.

Structured output

GBNF grammars, json_schema and response_format.

Tool calls

Jinja chat templates, with parsed and streamed tool-call deltas.

Context shift

A slot that fills its context drops the oldest tokens and keeps going.

Multimodal, no fetching

Image and audio as data: URIs only. The engine performs no I/O for a request.

Safetensors to GGUF

A numpy-only converter turns a Hugging Face Llama directory into GGUF on first load.

1	9	4	6
model per process	HTTP endpoints	prebuilt platforms	build targets: CPU to ROCm

Get started

```
$ npm ci && npm run build
$ ./build/onyx-engine -m model.gguf --port 8080
# add -np 4 for four concurrent slots
$ curl localhost:8080/v1/chat/completions \
 -H "Content-Type: application/json" \
 -d '{"messages": [{"role": "user",
    "content": "Hello!"}]}'
```

v0.4.0. Prebuilt for linux-x64, darwin-arm64, darwin-x64 and win32-x64.

github.com/JX-Holdings-LLC/Onyx-Engine